Introduction to NLP — Teaching Machines to Read

What is NLP?

Natural Language Processing — making computers understand the way humans talk and write.

Introduction to Natural Language Processing (NLP)

Think about how naturally you understand this sentence:

"I didn't say she stole the money."

Depending on which word you stress, it can have several different meanings.

• I didn't say she stole the money. → Someone else said it.

• I didn't say she stole the money. → I deny saying it.

• I didn't say she stole the money. → I may have implied it.

• I didn't say she stole the money. → Someone else stole it.

• I didn't say she stole the money. → Maybe she borrowed it.

• I didn't say she stole the money. → Perhaps she stole something else.

Humans understand these subtle differences naturally.

Now imagine teaching that to a computer.

That is exactly the challenge of Natural Language Processing (NLP) and one of the reasons it is one of the most exciting fields in Artificial Intelligence.

==========

Where Do You Use NLP Every Day?

1. Google Search

When you type:

```
"best pizza near me"
```

Google understands:

• best → quality preference

• pizza → food category

• near me → location intent

and returns relevant nearby restaurants.

---

2. Gmail Smart Reply

Suppose you receive:

```
"Can we meet tomorrow at 10 AM?"
```

Gmail may suggest:

• Sounds good!

• I'll check.

• See you tomorrow.

The system understands the email's context using NLP.

---

3. Autocorrect

You type:

```
teh
```

Your phone suggests:

```
the
```

NLP combined with statistical models predicts the word you intended to type.

---

4. Alexa, Siri, and Google Assistant

Voice Command:

```
"Set a timer for 10 minutes."
```

Process:

1. Speech is converted to text.
2. The text is analyzed.
3. The user's intent is identified.
4. The action is executed.

This entire pipeline relies heavily on NLP.

---

5. Spam Filters

Email:

```
"Congratulations! You won a FREE iPhone!"
```

The system analyzes the message and classifies it as:

```
Spam
```

using NLP-based text classification techniques.

---

6. ChatGPT

ChatGPT can:

• Understand questions

• Track conversation context

• Generate human-like responses

• Answer follow-up questions

All of this is powered by advanced NLP models.

==========

Why Is Human Language Hard for Computers?

Numbers are easy:

```
5 + 3 = 8
```

The answer is always the same.

Language, however, is messy and full of hidden meanings.

---

1. Ambiguity

Sentence:

```
"I saw the man with the telescope."
```

Possible meanings:

• I used a telescope to see the man.

• The man was holding a telescope.

Humans use context to determine the correct meaning.

Computers often struggle without additional information.

---

2. Sarcasm

Sentence:

```
"Oh great, another Monday."
```

Human interpretation:

```
Negative emotion
```

Computer interpretation (without context):

```
Positive emotion because of the word "great"
```

Understanding sarcasm remains a difficult NLP problem.

---

3. Context

Sentence:

```
"He kicked the bucket."
```

Literal meaning:

```
He physically kicked a bucket.
```

Actual meaning:

```
He died.
```

Humans easily understand idioms and expressions through context.

Computers must learn these patterns from large amounts of data.

---

4. Abbreviations and Slang

Examples:

• LOL = Laugh Out Loud

• BRB = Be Right Back

• GOAT = Greatest Of All Time

Language evolves constantly, making it difficult for NLP systems to stay current.

---

5. Multiple Languages

Example: chat

English: chat = conversation

French: chat = cat

Same spelling, completely different meaning.

NLP systems must identify the language before interpreting the word correctly.

==========

Key Challenges in NLP

1. Ambiguity

   Example: "I saw the man with the telescope"

2. Sarcasm

   Example: "Oh great, another Monday"

3. Context

   Example: "He kicked the bucket"

4. Slang and Abbreviations

   Examples: LOL, BRB, GOAT

5. Multiple Languages

   Example: chat (English vs French)

Real-Life Analogy — NLP as a Translator

Imagine you hired a brilliant translator who has read every book ever written but has never spoken to a human.

This translator:

• Understands grammar perfectly. • Can recognize language patterns. • Knows the meanings of millions of words. • Can predict what word is likely to come next. • Has read enormous amounts of text.

At first glance, this sounds like the perfect language expert.

However, there is one problem:

The translator has never experienced real human emotions, intentions, humor, or sarcasm.

For example:

Person A: "Oh great, another Monday."

A human immediately understands:

```
The speaker is probably unhappy.
```

The translator sees:

```
"great" = positive word
```

and may incorrectly assume the sentence expresses happiness.

Another example:

Person A: "Can you open the window?"

Humans understand:

```
This is a polite request.
```

The translator may interpret it literally as a question about someone's ability.

This is very similar to how modern NLP models work.

NLP models:

• Learn from massive amounts of text. • Identify patterns in language. • Understand grammar and sentence structure. • Predict likely meanings based on context. • Generate human-like responses.

But they do not truly experience:

• Happiness • Sadness • Humor • Sarcasm • Personal memories • Real-world consciousness

They learn statistical patterns rather than human experiences.

The Goal of NLP

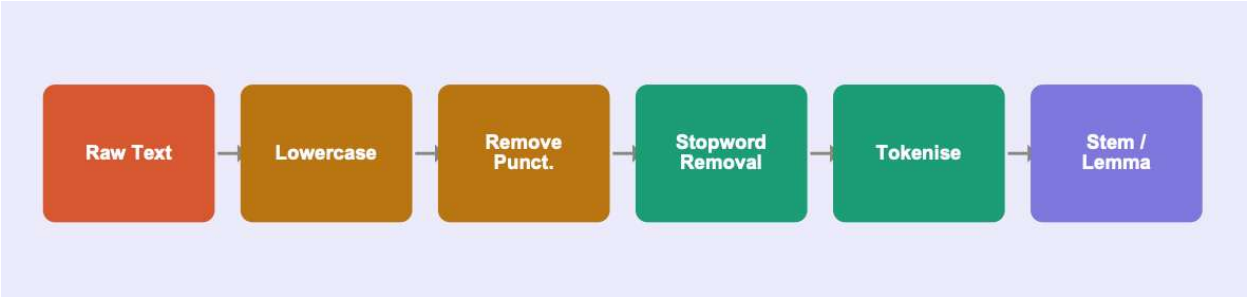
Our job as AI engineers is to give NLP systems the tools needed to bridge the gap between:

Human Language ↓ Understanding Context ↓ Understanding Intent ↓ Generating Meaningful Responses

The better the model understands context, emotions, and intent, the more natural its communication becomes.

Text Pre-processing Pipeline

Clean and normalise raw text before any model ever sees it. Raw text from the internet is messy: random capitalisation, emojis, HTML tags, punctuation everywhere, and filler words like 'the' and 'is' that carry no meaning. Just as you wash vegetables before cooking, you must clean text before feeding it to a model. Here is the standard 5-step pipeline:



Step 1 — Lowercasing

'Hello', 'HELLO', and 'hello' are the same word but a computer sees three different tokens. Converting everything to lowercase collapses them into one. Simple but critical.

```
# Lowercasing — always the first step
text = 'Hello! This is a SAMPLE Email to check  Manoj And Ram.  Are looking
# Python's built-in .lower() method
text_lower = text.lower()
print(text_lower)
```

```
hello! this is a sample email to check  manoj and ram.  are looking at screen
```

Step 2 — Removing punctuation & special characters

Punctuation like '!', '@', '#' usually adds no value for classification. We use regular expressions (regex) to strip them out.

```
import re
text = 'Hello! This is a SAMPLE Email to check  Manoj And Ram.  Are looking
text_clean = re.sub(r'[^a-z ]', '', text)
print(text_clean)
```

```
ello his is a  mail to check  anoj nd am  re looking at creen
```

Step 3 — Stopword removal

Stopwords are ultra-common words — 'the', 'is', 'a', 'and', 'in' — that appear everywhere but carry no distinctive meaning. Removing them reduces noise and shrinks your vocabulary. NLTK ships with a ready-made stopword list.

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords', quiet=True)
stop_words = set(stopwords.words('english'))
text = "I'm going to College today"
tokens = []
for word in text.split():
    if word not in stop_words:
        tokens.append(word)
print(tokens)
```

```
["I'm", 'going', 'College', 'today']
```

Step 4 — Tokenisation

Tokenisation means splitting a sentence into its individual units (tokens). A simple .split() works for most cases, but NLTK's word_tokenize handles edge cases like contractions: "don't" → ["do", "n't"]

```
import nltk
nltk.download('punkt_tab', quiet=True)
sentence = "I don't love spam emails, do you?"
# Method 1: basic split (misses contractions)
basic = sentence.split()
print('Basic split:', basic)

# Method 2: NLTK's smarter tokeniser
from nltk.tokenize import word_tokenize
smart = word_tokenize(sentence.lower())
```

```
print('NLTK tokenise:', smart)
```

```
Basic split: ['I', "don't", 'love', 'spam', 'emails,', 'do', 'you?']
NLTK tokenise: ['i', 'do', "n't", 'love', 'spam', 'emails', ',', 'do', 'you',
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Step 5 — Stemming vs Lemmatisation

'Running', 'runs', 'ran', 'runner' all refer to the same core concept. We need to reduce them to a single base form:

Technique	Rule-based?	'running' →	'better' →	'studies' →	Best for
Stemming	No — chops suffix	run	bett (wrong!)	studi	Speed; BoW models
Lemmatisation	Yes — uses grammar	run	good (correct)	study	Quality; NLP pipelines

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
nltk.download('wordnet', quiet=True)
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
words = ['running', 'studies', 'better', 'caring', 'happily']
for word in words:
  stem = stemmer.stem(word)
  lemma = lemmatizer.lemmatize(word, pos='v')
  print(f'{word:12} | stem: {stem:10} | lemma: {lemma}')
```

```
running      | stem: run        | lemma: run
studies      | stem: studi      | lemma: study
better       | stem: better     | lemma: better
caring       | stem: care       | lemma: care
happily      | stem: happili    | lemma: happily
```

Complete pipeline in one function

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

```
nltk.download(['stopwords', 'wordnet', 'punkt'], quiet=True)
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    # Step 1: Convert to lowercase
    text = text.lower()
    # Step 2: Remove punctuation and special characters
    text = re.sub(r'[^a-z ]', '', text)
    # Step 3: Tokenize
    tokens = text.split()
    # Step 4: Remove stopwords
    filtered_tokens = []
    for w in tokens:
        if w not in stop_words:
            filtered_tokens.append(w)

    tokens = filtered_tokens
    # Step 5: Lemmatize
    lemmatized_tokens = []
    for w in tokens:
        lemmatized_tokens.append(
            lemmatizer.lemmatize(w)
        )
    tokens = lemmatized_tokens
    # Convert list back to string
    return ' '.join(tokens)

# Test
print(preprocess('I am running to the stores to buy some groceries!'))
```

```
running store buy grocery
```

NOTE

Always lowercase BEFORE removing punctuation. Otherwise 'Hello!' and 'hello!' become different tokens after punctuation removal if you lowercase second. And always remove stopwords AFTER tokenisation — you need the word list first

Turning Text into Numbers

Computers only understand numbers — Bag of Words and TF-IDF convert text. After pre-processing you have clean words. But an ML model needs numbers. This section shows the two most important methods to convert a bag of words into a numeric matrix that any classifier can use.

Bag of Words (BoW) — just count words

Imagine you have 3 emails and a vocabulary of 5 words. BoW simply counts how many times each word appears in each email and builds a table:

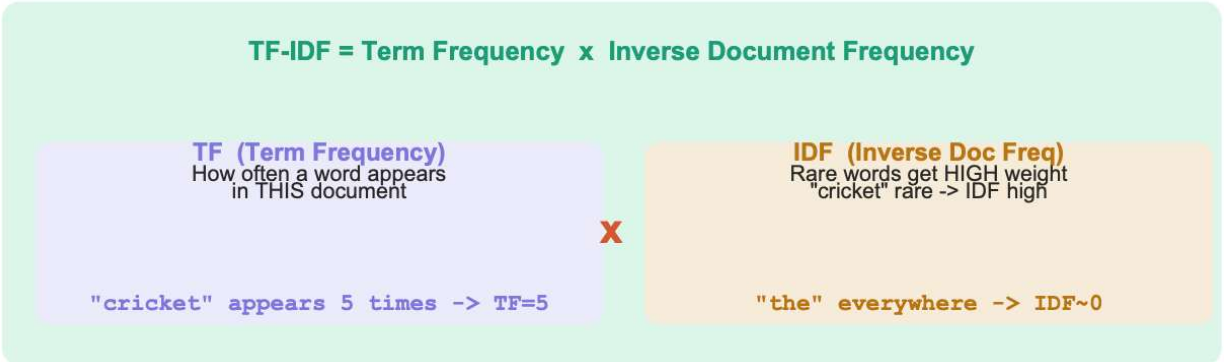
Email	cricket	match	free	win	money
Email 1 (cricket fan)	3	2	0	0	0
Email 2 (spam)	0	0	2	3	2
Email 3 (cricket spam)	2	1	1	2	1

```
from sklearn.feature_extraction.text import CountVectorizer
emails = [
'cricket match cricket match cricket', # genuine fan
'free win money free win money win', # spam
'cricket match free win money cricket', # cricket spam
]
# CountVectorizer builds the vocab and counts automatically
cv = CountVectorizer()
X = cv.fit_transform(emails).toarray() # .toarray() makes it readable
print('Vocabulary:', cv.get_feature_names_out())
print('BoW matrix:')
print(X)
```

```
Vocabulary: ['cricket' 'free' 'match' 'money' 'win']
BoW matrix:
[[3 0 2 0 0]
 [0 2 0 2 3]
 [2 1 1 1 1]]
```

TF-IDF — reward rare, important words

BoW treats 'the' and 'cricket' equally if they appear the same number of times. But 'the' is everywhere (useless) while 'cricket' in a spam email is very informative. TF-IDF fixes this by multiplying term frequency by how rare the word is across all documents:



```
from sklearn.feature_extraction.text import TfidfVectorizer
emails = [
'cricket match cricket match cricket',
'free win money free win money win',
'cricket match free win money cricket',
]
tfidf = TfidfVectorizer(sublinear_tf=True, max_df=0.95, min_df=1)
X_tfidf = tfidf.fit_transform(emails).toarray()
print('Feature names:', tfidf.get_feature_names_out())
print('TF-IDF matrix (rounded):')
import numpy as np
print(np.round(X_tfidf, 3))
```

```
Feature names: ['cricket' 'free' 'match' 'money' 'win']
TF-IDF matrix (rounded):
[[0.778 0.    0.628 0.    0.   ]
 [0.    0.532 0.    0.532 0.659]
 [0.646 0.382 0.382 0.382 0.382]]
```

BoW vs TF-IDF — when to use which?

Use BoW when: your vocabulary is small, you need speed, or you're building a quick prototype. Use TF-IDF when: some words are much more informative than others (spam detection, document classification, search ranking). TF-IDF almost always beats plain BoW.

Lab — Spam Email Classifier

Build a real spam detector using TF-IDF + Logistic Regression from scratch.

Part A — Load and explore the data

```
import pandas as pd
import numpy as np
!wget -q https://archive.ics.uci.edu/ml/machine-learning-databases/00228/sms
!unzip -o smsspamcollection.zip
!mv SMSSpamCollection SMSSpamCollection
df = pd.read_csv('SMSSpamCollection', sep='\t',
header=None, names=['label', 'message'])
# Quick look at the data
print(df.head())
print('\nShape:', df.shape) # (5574, 2)
print('\nClass balance:')
print(df['label'].value_counts()) # ham: 4827, spam: 747
print(f'Spam rate: {747/5574*100:.1f}%') # ~13% spam
# Check a spam example
print('\nSpam example:')
print(df[df['label']=='spam']['message'].iloc[0])
# 'Free entry in 2 a wkly comp to win FA Cup...'
```

```
Archive:  smsspamcollection.zip
  inflating: SMSSpamCollection
  inflating: readme
mv: 'SMSSpamCollection' and 'SMSSpamCollection' are the same file
  label                                                message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...

Shape: (5572, 2)

Class balance:
label
ham     4825
spam     747
Name: count, dtype: int64
Spam rate: 13.4%

Spam example:
Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA t
```

Part B — Pre-process and vectorise

```
import re, nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
nltk.download(['stopwords','wordnet'], quiet=True)
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
def preprocess(text):
  text = text.lower() # lowercase
  text = re.sub(r'[^a-z ]', '', text) # remove punctuation
  tokens = [w for w in text.split() if w not in stop_words] # stopwords
```

```
  tokens = [lemmatizer.lemmatize(w) for w in tokens] # lemmatise
  return ' '.join(tokens)
# Apply pre-processing to every message
df['clean'] = df['message'].apply(preprocess)
# Convert labels: spam=1, ham=0
df['target'] = (df['label'] == 'spam').astype(int)
# TF-IDF vectorisation
# max_features=5000 keeps only the top 5000 most informative words
# ngram_range=(1,2) also captures 2-word phrases like 'free win'
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2),
sublinear_tf=True)
X = tfidf.fit_transform(df['clean']) # sparse matrix (5574 x 5000)
y = df['target'].values
# Split: 80% train, 20% test, stratify keeps spam ratio balanced in both
X_tr, X_te, y_tr, y_te = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
print(f'Train: {X_tr.shape}, Test: {X_te.shape}')
```

```
Train: (4457, 5000), Test: (1115, 5000)
```

Part C — Train and evaluate

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report,
confusion_matrix, accuracy_score)
import seaborn as sns
import matplotlib.pyplot as plt
# Train Logistic Regression
# C=1.0 is regularisation strength (higher = less regularisation)
# max_iter=1000 ensures the model converges
clf = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
clf.fit(X_tr, y_tr) # this is where the model actually 'learns'
# Evaluate on test set (data the model has never seen)
y_pred = clf.predict(X_te)
print('Accuracy:', accuracy_score(y_te, y_pred))
# Accuracy: ~0.984 (98.4% correct!)
# Full report: precision, recall, F1
print(classification_report(y_te, y_pred,
target_names=['Ham (genuine)', 'Spam']))
# Confusion matrix
cm = confusion_matrix(y_te, y_pred)
print('Confusion matrix:')
print(cm)
# [[True Ham, False Spam],
# [Missed Spam, Caught Spam]]
```

```
Accuracy: 0.9659192825112107
              precision    recall  f1-score   support

Ham (genuine)       0.96      1.00      0.98       966
         Spam       1.00      0.74      0.85       149

     accuracy                           0.97      1115
```

```
       macro avg       0.98      0.87      0.92      1115
    weighted avg       0.97      0.97      0.96      1115

Confusion matrix:
[[966   0]
 [ 38 111]]
```

Part D — Predict new messages

```
def predict_spam(message):
  '''Predict if a new message is spam or ham.'''
  cleaned = preprocess(message) # same pipeline as training
  vector = tfidf.transform([cleaned]) # transform (not fit_transform!)
  pred = clf.predict(vector)[0]
  prob = clf.predict_proba(vector)[0]
  label = 'SPAM' if pred==1 else 'HAM'
  conf = prob[pred]
  return f'{label} (confidence: {conf:.1%})'
# Test on new messages
messages = [
'Congratulations! You won a free iPhone. Click here to claim now!',
'Hey, are we still meeting at 6pm for dinner?',
'URGENT: Your bank account has been suspended. Verify now!',
'Can you pick up milk on the way home?',
]
for msg in messages:
  print(f'{predict_spam(msg):30} | {msg[:55]}...')
# SPAM (confidence: 99.8%) | Congratulations! You won a free iPhone...
# HAM (confidence: 99.1%) | Hey, are we still meeting at 6pm...
# SPAM (confidence: 98.5%) | URGENT: Your bank account has been...
# HAM (confidence: 97.3%) | Can you pick up milk on the way home?
```

```
SPAM (confidence: 72.2%)       | Congratulations! You won a free iPhone. Clic
HAM (confidence: 96.9%)        | Hey, are we still meeting at 6pm for dinner?
HAM (confidence: 73.4%)        | URGENT: Your bank account has been suspended
HAM (confidence: 97.1%)        | Can you pick up milk on the way home?...
```

NLP Interview Questions and Answers

==========

Q1. What is Tokenisation and Why Is It Needed?

Answer:

Tokenisation is the process of splitting raw text into smaller units called tokens.

Tokens may be:

• Words

• Sub-words

• Characters

• Sentences

Example:

Text: "I love NLP"

Tokens: ["I", "love", "NLP"]

Why is tokenisation needed?

Machine Learning models cannot understand raw text directly.

They require text to be converted into structured units before creating numerical representations.

Benefits:

• Builds vocabulary mappings

• Enables word frequency counting

• Forms the foundation for text preprocessing

• Converts unstructured text into model-friendly format

Q2. What is the Difference Between Stemming and Lemmatization? Give an Example.

Answer:

Both techniques reduce words to their base form.

Stemming

• Uses simple rule-based chopping

• Removes prefixes or suffixes

• Does not check dictionary validity

• Faster

Examples:

running → run

playing → play

better → bett

Notice that "bett" is not a valid English word.

Lemmatization

• Uses vocabulary and grammar rules

• Considers context and part of speech

• Produces valid dictionary words

• Slower but more accurate

Examples:

running → run

playing → play

better → good

Key Difference

Stemming: Fast but may create invalid words.

Lemmatization: Slower but produces meaningful root words.

Q3. Why Do We Remove Stopwords? Can Removing Them Ever Be Harmful?

Answer:

Stopwords are common words that appear frequently in text.

Examples:

• the

• is

• am

• are

• and

• of

• in

These words usually carry little information and increase noise.

Benefits of removing stopwords:

• Reduces vocabulary size

• Improves processing speed

• Removes irrelevant information

• Improves model efficiency

Can it be harmful?

Yes.

Example:

Sentence: "This movie is not good."

If "not" is removed:

```
"This movie good"
```

The sentiment completely changes.

Therefore:

For Sentiment Analysis, Chatbots,

Question Answering,

and Language Understanding tasks,

stopword removal must be done carefully.

Q4. What Is TF-IDF and Why Is It Better Than Bag of Words?

Answer:

TF-IDF stands for:

TF = Term Frequency

IDF = Inverse Document Frequency

Formula:

:contentReference[oaicite:0]{index=0}

Term Frequency (TF)

Measures how often a word appears in a document.

Example:

Document: "cat cat dog"

TF(cat) = 2

TF(dog) = 1

Inverse Document Frequency (IDF)

Measures how rare a word is across all documents.

Common words get lower scores.

Rare words get higher scores.

Why TF-IDF is Better than Bag of Words

Bag of Words:

• Uses raw word counts

• Treats all words equally

TF-IDF:

• Rewards important words

• Penalizes common words

• Produces more meaningful features

Example:

Word: "the"

Appears in every document.

Bag of Words: High importance

TF-IDF: Low importance

Thus TF-IDF provides more informative representations.

Q5. What Evaluation Metric Would You Use for a Spam Classifier and Why?

Answer:

Accuracy alone can be misleading when classes are imbalanced.

Example:

Dataset:

87% Ham (Normal Emails) 13% Spam Emails

A model predicting everything as Ham gives:

Accuracy = 87%

But it detects zero spam.

Better Metrics

1. Precision

Measures:

```
Of emails predicted as spam,
how many were actually spam?
```

Formula:

:contentReference[oaicite:1]{index=1}

---

2. Recall

Measures:

```
Of all actual spam emails,
```

```
how many were detected?
```

Formula:

:contentReference[oaicite:2]{index=2}

---

3. F1 Score

Balances Precision and Recall.

Formula:

:contentReference[oaicite:3]{index=3}

Production Preference

Spam Detection usually prioritizes:

High Recall

Reason:

Missing spam emails is generally worse than accidentally flagging a few genuine emails.

Q6. How Does Logistic Regression Classify Text After TF-IDF Vectorisation?

Answer:

Step 1: Convert Text to TF-IDF Features

Example:

Email: "Win free iPhone now"

TF-IDF converts the email into a numeric vector.

Example:

[0.82, 0.67, 0.91, 0.54]

Step 2: Learn Feature Weights

Logistic Regression learns a weight for each word.

Example:

"free" → +2.4

"win" → +3.1

"offer" → +1.8

"meeting"→ -2.0

Positive weights indicate spam-like words.

Negative weights indicate normal-email words.

Step 3: Calculate Weighted Sum

The model computes:

$z = w_1x_1 + w_2x_2 + ... + b$

Step 4: Apply Sigmoid Function

:contentReference[oaicite:4]{index=4}

The sigmoid converts the score into a probability between 0 and 1.

Example:

P(spam) = 0.92

Step 5: Make Prediction

If:

P(spam) > 0.5

→ Spam

Else

→ Ham (Normal Email)

Summary:

Text

↓

Tokenisation

↓

Preprocessing

↓

TF-IDF Vectorisation

↓

Logistic Regression

↓

Weighted Sum

↓